

EXPENSER

Personal Finance Tracker

A Django-based Web Application for Income, Expense & Budget Management

Submitted By

Student Name	Student ID
Shihab Shaharia	2023100010136
Abid Ahmed Angkon	2023200010039
Mahfuz Iqram Sunny	2023200010034

PROJECT REPORT

This Report Presented in Partial Fulfillment of the Course

CSE474.3: Software Development and Project Management

in the Computer Science and Engineering Department

Submitted To: Md. Anower Perves (AP)

Lecturer, Department of CSE, Southeast University



SOUTHEAST UNIVERSITY

Dhaka, Bangladesh

June 6, 2026

Contents

1	Introduction	3
1.1	Project Goals	3
2	Project Planning	4
2.1	Development Methodology	4
2.2	Task Distribution	4
2.3	Tools Used for Planning	5
3	Environment Setup (Linux VM)	6
3.1	System Requirements	6
3.2	Step-by-Step Setup	6
3.2.1	1. Install System Dependencies	6
3.2.2	2. Set Up PostgreSQL	6
3.2.3	3. Clone the Repository	7
3.2.4	4. Create Virtual Environment and Install Dependencies	7
3.2.5	5. Configure Environment Variables	7
3.2.6	6. Run Migrations and Seed Data	7
3.2.7	7. Build Tailwind CSS and Start the Server	7
4	Project Structure	9
4.1	File Structure	9
4.2	Key Configuration Files	10
4.2.1	settings.py	10
4.2.2	URL Routing	11
5	Database Design	13
5.1	Models Overview	13
5.2	Entity-Relationship Diagram	14
6	System Features and Implementation	15
6.1	Authentication	15
6.2	CRUD Operations	15

6.3	Forms and Validation	16
6.4	Dashboard	16
6.5	Reports and CSV Export	17
6.6	Custom Template Tags	17
7	Team Contributions	18
7.1	Shihab Shaharia — Person 1	18
7.2	Abid Ahmed Angkon — Person 2	18
7.3	Mahfuz Iqram Sunny — Person 3	19
8	Conclusion	20

Chapter 1

Introduction

Expenser is a web-based personal finance management application developed as part of the CSE474.3 Software Development and Project Management Lab course at Southeast University. The main idea behind this project was to build something useful while learning Django properly. We wanted to make a simple tool where a user can log their daily income and expenses, keep track of monthly budgets, and see reports on how they are spending.

The application is built using Python and Django on the backend, PostgreSQL as the database, and Tailwind CSS for the frontend styling. It runs on a Linux virtual machine environment during development.

1.1 Project Goals

- Allow users to record income and expense entries with categories and tags
- Let users set monthly budgets per category and track spending
- Generate financial reports over different time periods
- Export report data as CSV
- Implement Django's authentication system for secure user accounts
- Practice team-based development using Git and GitHub

Chapter 2

Project Planning

2.1 Development Methodology

We used an Agile approach for this project. Since we were three people working on different parts at the same time, Agile made sense because we could work in parallel and merge our work regularly through GitHub pull requests. A strict Waterfall approach would not have worked well here because we needed to adjust things as we went.

We split the work into three rough sprints, each roughly one to two weeks:

Sprint	Who	What was done
1	Shihab	Project setup, database models, admin, authentication, base templates, category seeding
2	Abid	Entry, Category, Budget views and forms, filtering, pagination
3	Mahfuz	Reports module, CSV export, custom template tags, Chart.js charts

Table 2.1: Sprint plan

2.2 Task Distribution

We divided the work based on Django’s natural structure. Each person owned one layer of the application:

- **Shihab Shaharia** – Project structure, models, authentication, admin, dashboard, UI templates
- **Abid Ahmed Angkon** – All CRUD views and forms for entries, categories, and budgets

- **Mahfuz Iqram Sunny** – Reports, CSV export, custom template tags, charts

We used GitHub for version control. Each person worked on their own branch and submitted pull requests when features were ready. The main branch was kept stable at all times.

2.3 Tools Used for Planning

- GitHub Issues and PRs for task tracking and code review
- WhatsApp group for quick communication
- Shared Google Doc for initial planning notes
- draw.io for the ER diagram draft

Chapter 3

Environment Setup (Linux VM)

The project runs on a Linux virtual machine (Ubuntu 22.04). Below are the steps to set it up from scratch.

3.1 System Requirements

- Ubuntu 22.04 LTS (or similar Debian-based Linux)
- Python 3.11 or 3.12
- PostgreSQL 14 or later
- Node.js 18 or 20 LTS (for Tailwind CSS build)
- Git

3.2 Step-by-Step Setup

3.2.1 1. Install System Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv
sudo apt install -y postgresql postgresql-contrib
sudo apt install -y nodejs npm git
```

3.2.2 2. Set Up PostgreSQL

```
sudo -u postgres psql
```

Inside the PostgreSQL shell:

```
CREATE DATABASE expenser_db;  
CREATE USER expenser_user WITH PASSWORD 'expenser_pass';  
GRANT ALL PRIVILEGES ON DATABASE expenser_db TO expenser_user;  
\q
```

3.2.3 3. Clone the Repository

```
git clone https://github.com/shihabshaharia/expenser.git  
cd expenser
```

3.2.4 4. Create Virtual Environment and Install Dependencies

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

3.2.5 5. Configure Environment Variables

Create a `.env` file in the project root:

```
DJANGO_SECRET_KEY=your-secret-key-here  
DEBUG=True  
ALLOWED_HOSTS=localhost,127.0.0.1  
DB_NAME=expenser_db  
DB_USER=expenser_user  
DB_PASSWORD=expenser_pass  
DB_HOST=localhost  
DB_PORT=5432
```

3.2.6 6. Run Migrations and Seed Data

```
python manage.py migrate  
python manage.py createsuperuser  
python manage.py seed_categories
```

3.2.7 7. Build Tailwind CSS and Start the Server

```
python manage.py tailwind build  
python manage.py runserver
```

The application will be available at <http://127.0.0.1:8000>.

Chapter 4

Project Structure

4.1 File Structure

The project follows Django's standard app-based layout. Each feature has its own Django app:

```
expenser/                                <- Django project config
    settings.py
    urls.py
    wsgi.py

accounts/                                <- User registration, login, signals
    models.py
    views.py
    urls.py
    signals.py
    forms.py

core/                                     <- Categories, Tags, Entries, Dashboard
    models.py
    views.py
    forms.py
    urls.py
    dashboard_urls.py
    templatetags/
        expenser_tags.py    <- Custom filters and template tags
    management/commands/
        seed_categories.py
        process_recurring.py

budgets/                                 <- Budget model and CRUD
```

```
models.py
views.py
forms.py
urls.py

reports/                                <- Reports and CSV export
    views.py
    urls.py

templates/                              <- All HTML templates
    base.html
    core/
    budgets/
    reports/
    registration/
    accounts/

theme/                                  <- Tailwind CSS build pipeline
    static_src/
        tailwind.config.js
```

4.2 Key Configuration Files

4.2.1 settings.py

The settings file controls how Django is configured. Some important sections:

```
INSTALLED_APPS = [
    # Django built-ins
    'django.contrib.admin',
    'django.contrib.auth',
    ...
    # Third party
    'tailwind',
    # Our apps
    'theme', 'accounts', 'core', 'budgets', 'reports',
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
```

```

        'NAME': os.environ.get('DB_NAME'),
        'USER': os.environ.get('DB_USER'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}

AUTH_USER_MODEL = 'accounts.User'

LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/accounts/login/'

```

All sensitive values (secret key, database password) are loaded from a `.env` file using `python-dotenv` so they are not hardcoded in the source code.

4.2.2 URL Routing

Django routes HTTP requests using `urls.py` files. The top-level URL config in `expenser/urls.py` distributes requests to each app:

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),
    path('entries/', include('core.urls')),
    path('budgets/', include('budgets.urls')),
    path('reports/', include('reports.urls')),
    path('', include('core.dashboard_urls')),
]

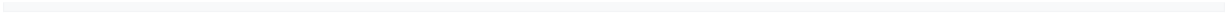
```

Each app then has its own `urls.py`. For example, `core/urls.py`:

```

urlpatterns = [
    path('', EntryListView.as_view(), name='entry_list'),
    path('add/', EntryCreateView.as_view(), name='entry_create'),
    path('<int:pk>/edit/', EntryUpdateView.as_view(), name='entry_update'),
    path('<int:pk>/del/', EntryDeleteView.as_view(), name='entry_delete'),
    path('categories/', CategoryListView.as_view(), name='category_list'),
    ...
]

```



Chapter 5

Database Design

5.1 Models Overview

The application has five main models. All models filter data by the logged-in user, so one user never sees another user's data.

Model	Purpose
User	Custom user model extending Django's AbstractUser, adds unique email
Category	Income or expense category, belongs to a user
Tag	Freeform label that can be attached to multiple entries
Entry	A single income or expense transaction
Budget	A monthly spending limit for one category

Table 5.1: Application models

5.2 Entity-Relationship Diagram

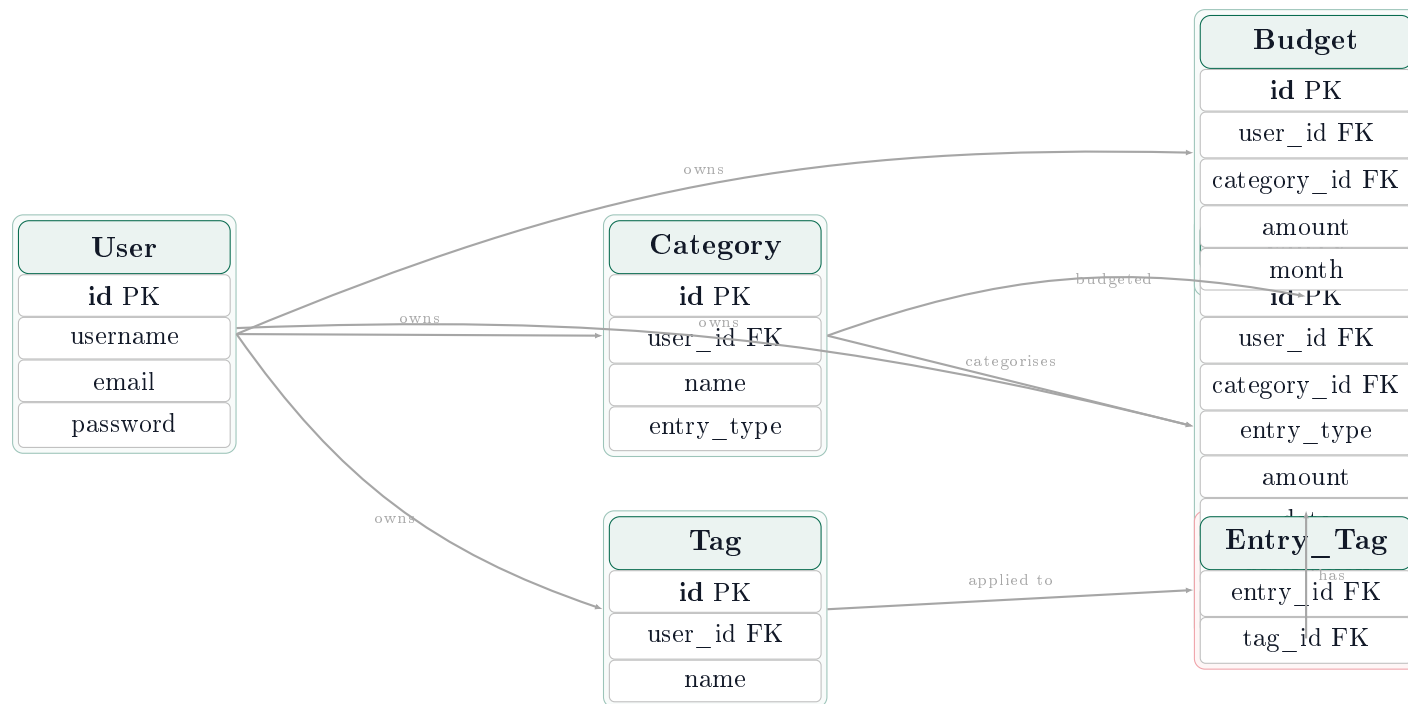


Figure 5.1: Entity-Relationship Diagram

Chapter 6

System Features and Implementation

6.1 Authentication

User registration and login are handled using Django's built-in auth system. We created a custom `User` model in the `accounts` app that extends `AbstractUser` and adds a unique email field.

When a new user registers, a Django signal automatically runs and creates 14 default categories (9 expense, 5 income) for them. This is done through a post-save signal in `accounts/signals.py` that calls a utility function from `core/utils.py`.

6.2 CRUD Operations

All the main features (entries, categories, budgets) follow the same pattern using Django's Class-Based Views (CBVs). Here is how the entry CRUD works as an example:

```
class EntryListView(LoginRequiredMixin, ListView):
    model = Entry
    context_object_name = 'entries'
    paginate_by = 25

    def get_queryset(self):
        # Always filter by the logged-in user
        return Entry.objects.filter(user=self.request.user)

class EntryCreateView(LoginRequiredMixin, CreateView):
    model = Entry
    form_class = EntryForm
    success_url = reverse_lazy('entry_list')

    def form_valid(self, form):
```

```
# Attach the current user before saving
form.instance.user = self.request.user
return super().form_valid(form)

class EntryDeleteView(LoginRequiredMixin, UserPassesTestMixin,
    DeleteView):
    model = Entry

    def test_func(self):
        # Make sure only the owner can delete
        return self.get_object().user == self.request.user
```

Listing 6.1: Entry views using Django CBVs

The same pattern is used for categories and budgets. Every queryset filters by `user=request.user` to make sure users can only see their own data.

6.3 Forms and Validation

We used Django’s `ModelForm` for all input forms. The `EntryForm` does the following validation:

- Amount must be greater than zero
- Date cannot be in the future
- The selected category’s type must match the entry type (e.g. an income entry cannot use an expense category)
- Tags are entered as comma-separated text and saved as M2M relationships

The `BudgetForm` handles a special case: the HTML `<input type="month">` widget sends `YYYY-MM` but Django’s `DateField` expects `YYYY-MM-DD`. The form’s `clean_month()` method adds `-01` to fix this.

6.4 Dashboard

The dashboard uses Django’s ORM to calculate monthly totals:

```
total_income = Entry.objects.filter(
    user=user,
    date__year=today.year,
    date__month=today.month,
```

```
entry_type='INCOME')
).aggregate(total=Sum('amount'))['total'] or 0
```

It also generates data for two charts: a 30-day daily trend chart and an expenses-by-category donut chart. These are passed to the template as JSON and rendered using Chart.js.

6.5 Reports and CSV Export

The reports page allows users to filter by time period (this week, this month, last month, this year, custom range). The `ReportView` calculates totals for the selected period and passes them to a Chart.js bar chart.

The CSV export uses Python's built-in `csv` module:

```
class ExportCSVView(LoginRequiredMixin, View):
    def get(self, request):
        response = HttpResponse(content_type='text/csv')
        response['Content-Disposition'] = 'attachment; filename="
            report.csv"'
        writer = csv.writer(response)
        writer.writerow(['Date', 'Type', 'Category', 'Amount', '
            Description'])
        for entry in entries:
            writer.writerow([entry.date, entry.entry_type,
                            entry.category.name, entry.amount,
                            entry.description])
        return response
```

6.6 Custom Template Tags

We created a custom template tag library in `core/templatetags/expenser_tags.py`. This allows us to use filters directly in Django templates:

- `currency` – formats a number as Tk. 1,234.56
- `percentage` – calculates percentage safely (avoids division by zero)
- `tag_pills` – renders coloured badge tags for each entry
- `category_color_hex` – returns a colour based on the category name for avatar backgrounds

Chapter 7

Team Contributions

Name	Student ID	Role
Shihab Shaharia	2023100010136	Person 1
Abid Ahmed Angkon	2023200010039	Person 2
Mahfuz Iqram Sunny	2023200010034	Person 3

7.1 Shihab Shaharia — Person 1

- Set up the Django project, folder structure, PostgreSQL database, and virtual environment configuration
- Designed all database models and wrote the initial migrations
- Set up Django admin with list displays and filters for all models
- Built the authentication system including the custom User model and registration view
- Wrote the post-save signal that seeds default categories for new users
- Built the dashboard view with ORM aggregations and chart data
- Created all base HTML templates and the overall page layout
- Managed the GitHub repository, reviewed pull requests, and resolved merge conflicts

7.2 Abid Ahmed Angkon — Person 2

- Implemented all Entry views (list, create, update, delete) using Django CBVs

- Built the EntryForm with field-level and cross-field validation
- Implemented all Category views and CategoryForm
- Built all Budget views with spending calculation logic
- Handled the BudgetForm month field coercion from YYYY-MM to a date
- Added entry filtering by type, category, tag, and date range
- Wrote the `process_recurring` management command
- Wired up the slide-over drawer for add/edit forms

7.3 Mahfuz Iqram Sunny — Person 3

- Built the ReportView with all date range presets and ORM aggregations
- Implemented the CSV export using Python's built-in csv module
- Wrote all custom template tags: `currency`, `percentage`, `tag_pills`, `category_color_hex`
- Set up Chart.js for the area trend chart, donut chart, and sparkline cards on the dashboard
- Designed and styled the reports page template
- Helped with UI adjustments across the entry list and category pages

Chapter 8

Conclusion

We built Expenser as a working personal finance tracker using Django and PostgreSQL. The project helped us understand how to structure a real Django application with multiple apps, how to use class-based views properly, how forms and validation work, and how to use signals and custom management commands.

Working as a team through GitHub taught us how to manage branches, review each other's code, and merge work without breaking things. We ran into a few issues along the way, like resolving merge conflicts and debugging template rendering problems, but we sorted them out as we went.

The application covers everything that was required: user authentication, transaction tracking, category and budget management, reports, and CSV export. It runs on a Linux virtual machine and the setup process is documented in this report.

Repository: <https://github.com/shihabshaharia/expenser>

Bibliography

- [1] Django Software Foundation, *Django Documentation*, version 5.2. <https://docs.djangoproject.com>
- [2] PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs>
- [3] Tailwind Labs, *Tailwind CSS Docs*. <https://tailwindcss.com/docs>
- [4] Chart.js Contributors, *Chart.js Docs*. <https://www.chartjs.org/docs>